

网络安全 - 端口扫描检测实验报告

实验目的

本实验旨在通过对 NetFlow 网络测量数据分析，识别潜在的恶意流量并模拟在线算法检测恶意远程主机。具体包括：

1. 分析 TCP SYN 扫描攻击
2. 利用已知危险端口进行端口扫描检测
3. 实现并评估 Bro 的端口扫描检测算法

实验原理

TCP SYN 扫描检测

TCP SYN 扫描是一种常见的网络攻击手段，攻击者向许多不同目的地发送 TCP SYN 包以发现网络中的脆弱机器。在正常的 TCP 连接中，连接建立过程包含三次握手：SYN、SYN-ACK 和 ACK。而在 SYN 扫描中，攻击者只发送 SYN 包，如果收到 SYN-ACK 响应则继续完成连接，否则就标记为目标不可达。

Bro 端口扫描检测算法

Bro 系统的端口扫描检测算法基于这样的直觉：失败的连接尝试（仅 SYN 或 SYN/RST 流）是识别扫描的良好指标。该算法区别对待不同服务的连接尝试，对于一组"良好服务"端口（如 HTTP 80、SSH 22 等），只跟踪失败的连接尝试；而对于其他服务，则跟踪所有连接尝试。当某个发送主机对应的被跟踪连接尝试的目标地址数量超过阈值 T 时，Bro 将该主机标记为扫描器。

实验内容与结果分析

Part A: TCP SYN 扫描检测

首先我们探索 NetFlow 跟踪数据，确定普林斯顿大学校园网络观察到的显著的仅有 TCP SYN 标志的流量（即仅有 SYN 包而没有 ACK 包的流）。

通过分析 netflow.csv 数据集，我们计算出 TCP 流中仅有 SYN 包而没有 ACK 包的比例：

```
tcp_flows = 0
syn_only_flows = 0

for flow in netflow_data:
    if flow["Protocol"] == "TCP":
        tcp_flows += 1
        flags = flow["Flags"]
        # 检查是否有SYN标志但没有ACK标志
        if "S" in flags and "A" not in flags:
            syn_only_flows += 1

percent_synonly = (syn_only_flows / float(tcp_flows)) * 100
```

这段代码遍历所有网络流数据，统计 TCP 协议的流总数以及其中只包含 SYN 标志而不包含 ACK 标志的流数量。通过检查 Flags 字段中是否包含"S"（SYN 标志）且不包含"A"（ACK 标志）来识别潜在的扫描流量。

实验结果：

- 总 TCP 流数：73494
- 仅 SYN 流数：64993
- 仅 SYN 流占比：88.43%

这个结果表明，在我们的数据集中绝大多数 TCP 流都是未完成的连接尝试，这可能意味着存在大量的端口扫描活动。

Part B: 基于已知危险端口的端口扫描检测

我们将以下端口视为已知危险端口：135（Windows RPC）、139（NetBIOS）、445（SMB）和 1433（SQL-Snake）。这些端口对应常见的 Windows 漏洞攻击。

我们将所有 TCP 流分为四类：

1. 到已知危险端口的 SYN-only 流
2. 到已知危险端口的其他流
3. 到其他端口的 SYN-only 流
4. 到其他端口的其他流

```

bad_ports = ["135", "139", "445", "1433"]

# 初始化计数器
syn_only_to_bad_ports = 0
other_to_bad_ports = 0
syn_only_to_other_ports = 0
other_to_other_ports = 0

# 分类所有TCP流
for flow in netflow_data:
    if flow["Protocol"] == "TCP":
        dst_port = flow["Dst port"]
        flags = flow["Flags"]

        # 检查是否为SYN-only流（有SYN无ACK）
        is_syn_only = "S" in flags and "A" not in flags

        # 检查目标端口是否为已知危险端口
        if dst_port in bad_ports:
            if is_syn_only:
                syn_only_to_bad_ports += 1
            else:
                other_to_bad_ports += 1
        else:
            if is_syn_only:
                syn_only_to_other_ports += 1
            else:
                other_to_other_ports += 1

```

该代码逻辑清晰地分类统计了各种类型的网络流。首先定义危险端口列表，然后遍历所有 TCP 流，根据目标端口是否为危险端口以及是否为 SYN-only 流将流量分为四类进行统计。这样可以分别计算不同类型流量的比例，从而更好地识别恶意扫描行为。

实验结果：

- 流向已知危险端口的流占有所有 TCP 流的比例：5.90%
- SYN-only 流占有所有流向已知危险端口的流的比例：99.79%
- SYN-only 流占有所有流向其他端口的流的比例：87.72%

从结果可以看出，流向已知危险端口的流量相对较少，但其中绝大部分都是 SYN-only 流，这强烈表明这些流量是恶意扫描行为。

恶意主机与良性主机分析

为了进一步识别恶意主机，我们找出以下三类主机：

1. 恶意主机：发送 SYN-only 流或发送到已知危险端口流的外部主机
2. 良性主机：发送非 SYN-only 流到其他（非危险）端口的外部主机
3. 可疑主机：同时出现在恶意主机和良性主机集合中的主机

```

# InternalIP returns true if the argument IP address is within the Princeton network
def internalIP(ip):
    s = ip.split('.')
    if s[0] == "128" and s[1] == "112":
        return True
    return False

# 初始化集合
malicious_hosts = set()
benign_hosts = set()

# 遍历所有TCP流，分类主机
for flow in netflow_data:
    if flow["Protocol"] == "TCP":
        src_ip = flow["Src IP addr"]

        # 只考虑来自普林斯顿网络外部的源IP
        if not internalIP(src_ip):
            dst_port = flow["Dst port"]
            flags = flow["Flags"]

            # 检查是否为SYN-only流
            is_syn_only = "S" in flags and "A" not in flags

            # 检查目标端口是否为已知危险端口
            is_known_bad_port = dst_port in bad_ports

            # 添加到恶意主机集：发送SYN-only流或发送到已知危险端口的流
            if is_syn_only or is_known_bad_port:
                malicious_hosts.add(src_ip)

            # 添加到良性主机集：发送非SYN-only流到其他（非危险）端口的流
            if not is_syn_only and not is_known_bad_port:
                benign_hosts.add(src_ip)

# 找出可疑主机（同时在恶意和良性集合中的主机）
questionable_hosts = malicious_hosts.intersection(benign_hosts)

# 从恶意和良性主机集中移除可疑主机
malicious_hosts = malicious_hosts.difference(questionable_hosts)
benign_hosts = benign_hosts.difference(questionable_hosts)

# 计算数量

```

```
num_malicious_hosts = len(malicious_hosts)
num_benign_hosts = len(benign_hosts)
num_questionable_hosts = len(questionable_hosts)
```

这段代码的核心思想是通过主机行为模式来分类主机。首先定义一个函数判断 IP 是否属于内部网络，然后遍历所有 TCP 流，将外部主机根据其流量特征分为恶意主机和良性主机。通过集合操作找出可疑主机（即同时出现在两个集合中的主机），并将它们从两个集合中移除，最终得到纯净的恶意主机和良性主机集合。

实验结果：

- 剩余恶意主机数：4479
- 剩余良性主机数：66
- 可疑主机数：12

Part C: 在线端口扫描检测

Bro 端口扫描检测算法实现

我们实现了 Bro 的端口扫描检测算法：

```

from collections import defaultdict

class Bro:
    # class constructor saves the parameter threshold value and sets instance variables
    def __init__(self, threshold):
        self.T = threshold

        # self.good_services is the list of port numbers to which successful connections
        # (SYN and ACK) should not be tracked
        self.good_services = [80, 22, 23, 25, 113, 20, 70]

        # self.tracked maps sending hosts to a set of destination addresses
        # from tracked connection attempts
        self.tracked = defaultdict(set)

    # block_connection() takes a sending host IP address of an incoming connection
    # returns True if the connection should be blocked (because the host has surpassed
    # the threshold of destination addresses in tracked connection attempts) or False otherwise
    def block_connection(self, host_ip):
        # 检查主机是否已经跟踪了超过阈值的目的地数量
        if len(self.tracked[host_ip]) >= self.T:
            return True
        return False

    # process_flow() takes a netflow record of a TCP flow that was not blocked
    # and should update self.tracked with the contents of the record
    def process_flow(self, netflow_record):
        src_ip = netflow_record["Src IP addr"]
        dst_ip = netflow_record["Dst IP addr"]
        dst_port = int(netflow_record["Dst port"])
        flags = netflow_record["Flags"]

        # 检查这是否是一个失败的连接尝试(SYN-only 或 SYN/RST)
        is_failed_attempt = ("S" in flags and "A" not in flags) or ("R" in flags and "S" in flags)

        # 如果目标端口是"good_services"之一
        if dst_port in self.good_services:
            # 只跟踪失败的连接尝试
            if is_failed_attempt:
                self.tracked[src_ip].add(dst_ip)
        else:

```

```
# 跟踪所有连接尝试
self.tracked[src_ip].add(dst_ip)
```

1. **init** 方法初始化阈值 T 和"良好服务"端口列表，使用 defaultdict(set) 来存储每个主机的目标地址集合
2. **block_connection** 方法检查主机是否应该被阻止，当主机跟踪的目标地址数量达到阈值时返回 True
3. **process_flow** 方法处理网络流记录，根据端口类型和连接状态决定是否跟踪该连接

特别注意的是，对于"良好服务"端口（如 HTTP、SSH 等），只跟踪失败的连接尝试（SYN-only 或 SYN/RST），而对于其他端口则跟踪所有连接尝试。这种差异化处理提高了算法的准确性。

算法敏感性分析

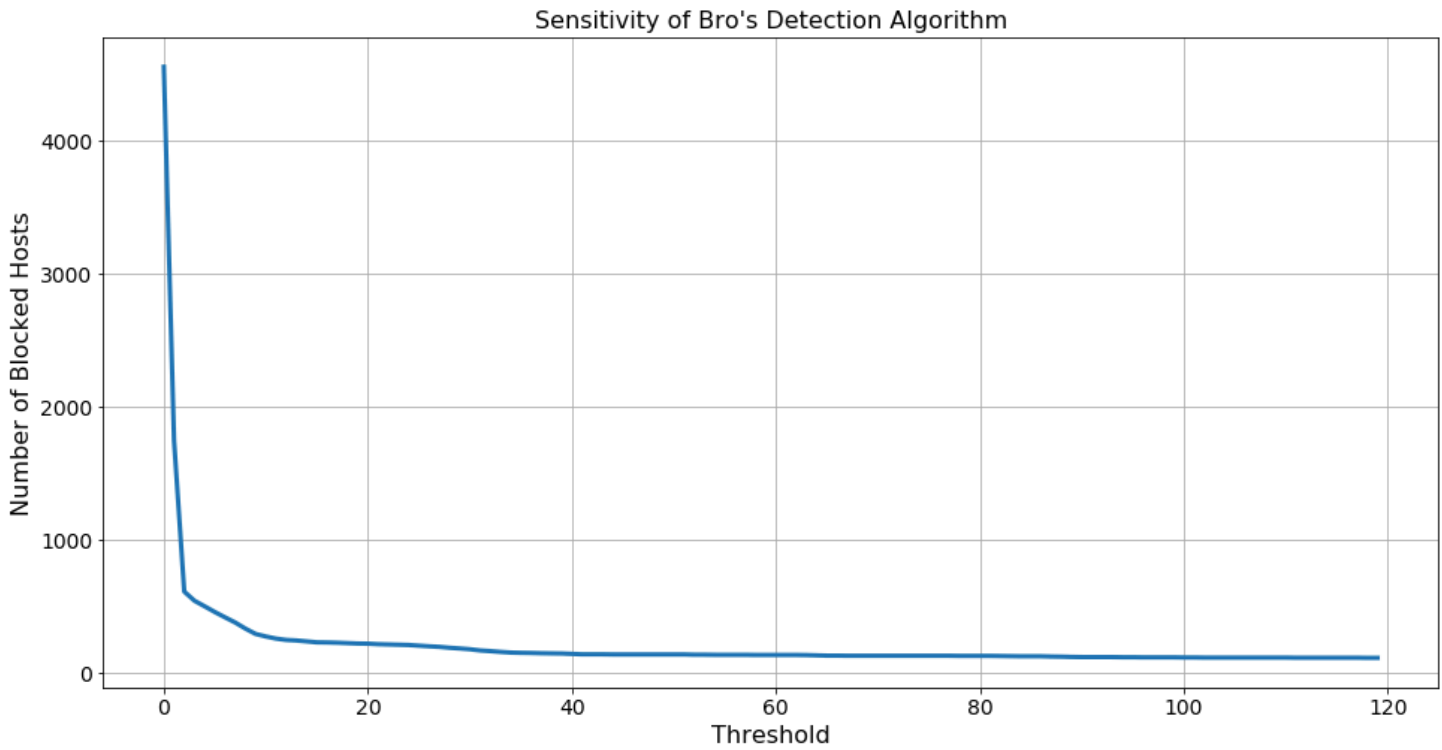
我们分析了 Bro 算法对阈值参数 T 的敏感性，创建了一个 120 元素的列表，其中元素 i 包含由 `run_bro(i)` 返回的被阻止主机数量：

```
def run_bro(threshold):
    blocked_hosts = set()
    bro = Bro(threshold)
    for flow in netflow_data:
        src_ip = flow["Src IP addr"]
        if flow["Protocol"] != "TCP":
            continue
        if internalIP(src_ip):
            continue
        block = bro.block_connection(src_ip)
        if block:
            blocked_hosts.add(src_ip)
            continue
        bro.process_flow(flow)
    return blocked_hosts

blocked_hosts_counts = []
for threshold in range(120):
    blocked_hosts = run_bro(threshold)
    blocked_hosts_counts.append(len(blocked_hosts))
```

`run_bro` 函数模拟了在线检测过程，按照时间顺序处理每个网络流。对于每个流，首先检查是否为 TCP 协议且来自外部网络，然后检查是否应该阻止该主机的连接，如果应该阻止就将其加入阻止列表，否则就处理该流记录。通过遍历不同阈值，我们可以分析算法的敏感性。

结果如下：



实验总结

通过本次实验，我们深入理解了网络端口扫描检测的原理和方法。实验结果表明：

1. 在网络流量中存在大量未完成的 TCP 连接（SYN-only 流），占比高达 88.43%，这很可能是端口扫描活动造成的。
2. Bro 的在线检测算法在阈值为 100 时能够较好地平衡检测效果和误报率，但相比离线分析方法，其检测到的恶意主机会更少。
3. 网络安全防护需要综合运用多种检测方法，既要通过离线分析全面识别威胁，也要通过在线检测实时阻止攻击。